

闲鱼店铺运营

自动化架构

内容侧全自动 · 客服侧人机协同
7 大模块 · 4 周落地 · 月成本 < ¥50

出品:由 Kiro 协助整理

闲鱼店铺运营 · AI 全自动化架构

目标：从选品 → 文案 → 主图 → 上架 → 客服 → 引流 → 复购 → 数据分析，**全链路 AI 自动跑。**

人只做三件事：审核、放行、收钱。

核心引擎：AI Agent + 浏览器自动化 + 闲鱼/微信对接 + 本地任务调度。

一、一句话看懂这套系统

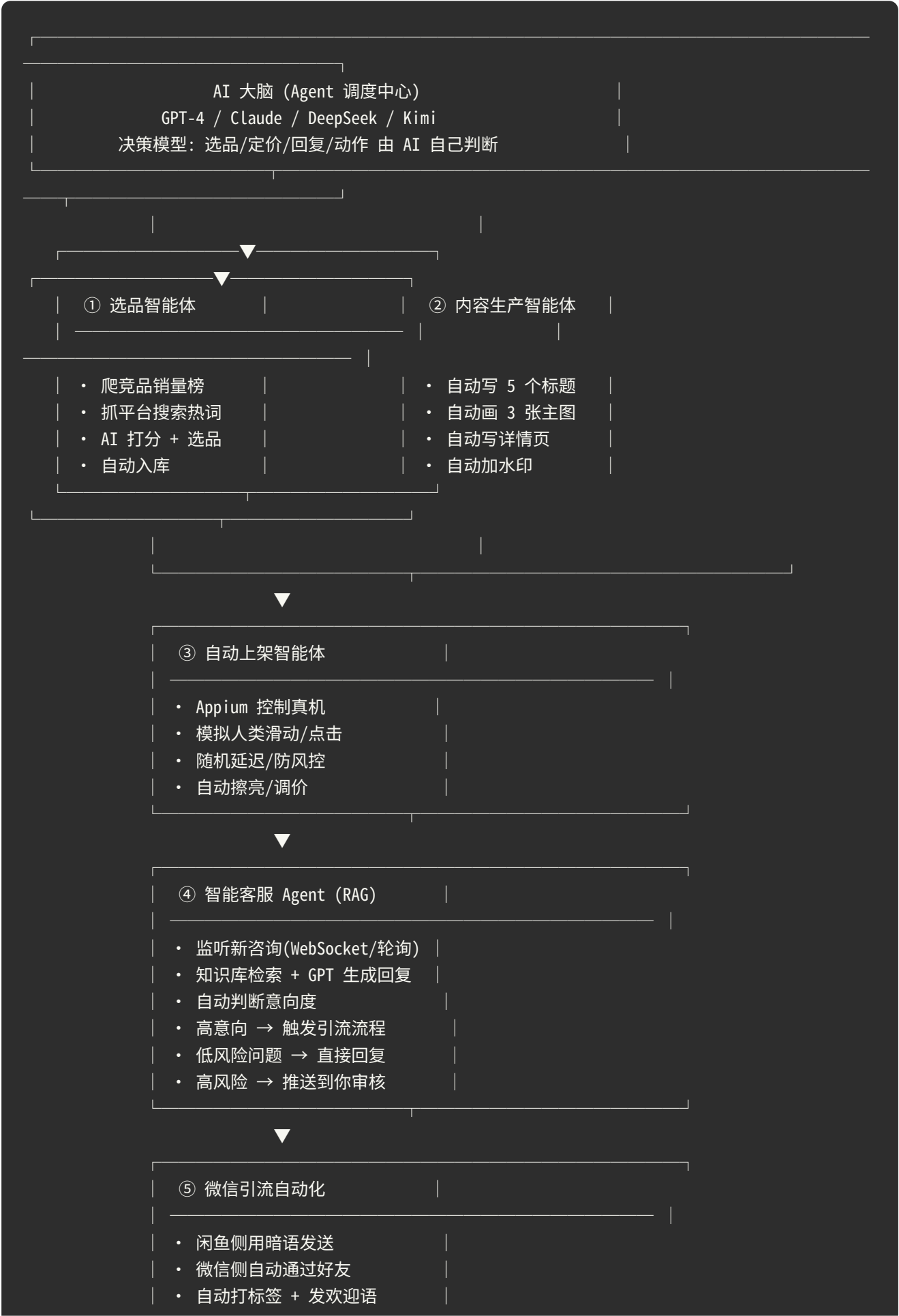
你只需要：早上喝咖啡，看一眼昨晚 AI 做了多少单。

系统自己：选品、写文案、画主图、发商品、回咨询、加微信、跟进客户、出周报。

人工保留环节（红线，不能省）： - 报价决策 / 议价（涉及钱必须你点头） - 复杂方案沟通（客户问技术细节时） - 高风险操作放行（首次给某客户发微信号）

其他全部 AI 自动。

二、整体架构（全自动版）





三、技术栈（全自动落地）

层级	技术选型	说明
AI 大脑	GPT-4o / Claude 3.5 / DeepSeek-V3	主决策 + 文案生成
图像生成	Stable Diffusion + ComfyUI / DALL · E 3	主图自动生成
Agent 框架	LangChain / LangGraph / AutoGen	多智能体调度
闲鱼操作	Appium + 真机 / 安卓模拟器	模拟人工
微信操作	企业微信 API + ItChat (慎用)	引流 + 客服
任务调度	Celery + Redis / APScheduler	定时任务
数据库	PostgreSQL + Redis + 向量库(Chroma)	客户/SKU/会话
后端	Python + FastAPI	业务接口
前端面板	Vue3 / React (你看监控用)	实时大屏
部署	Docker + 单台云服务器(2核4G)	¥80/月

四、各智能体详细设计

智能体 ① 选品 Agent（自动找爆款）

核心能力：每天自动从闲鱼、淘宝、抖音找出"高需求 + 你能做"的服务方向。

工作流程：

```
class SourcingAgent:
    def daily_scan(self):
        # 1. 爬取闲鱼搜索热词 Top 100
        hot_keywords = self.crawl_xianyu_hot()

        # 2. 爬取竞品销量数据（销量>50 的近期上架）
        competitors = self.crawl_competitors([
            "数据采集", "爬虫", "自动化",
            "Python脚本", "小程序定制", "AI"
        ])

        # 3. AI 打分（结合你的能力清单）
        scored = self.llm.score(
            candidates=competitors,
            criteria={
                "你的服务清单": SERVICE_LIST,
                "评分维度": ["市场需求", "竞争激烈度", "客单价", "交付难度"]
            }
        )

        # 4. 自动入库 Top 10
        new_skus = scored.top(10)
        for sku in new_skus:
            self.db.insert(sku)
            self.notify(f"今日新增 SKU: {sku.title}")
```

关键词库自动更新： - 每天爬 5118、闲鱼热搜、百度指数 - AI 自动剔除违禁词（"刷单""破解"等） - 自动按转化率排序

人工介入：仅在 AI 提示"高利润高风险"项目时弹窗确认。

智能体 ② 内容生产 Agent（自动写文案 + 画图）

输入：一个 SKU **输出：**5 个标题 + 3 张主图 + 1 篇详情页 + 30 条 FAQ

Prompt 工程示例：

```
TITLE_PROMPT = """
```

```
你是一名闲鱼爆款标题撰写专家。
```

```
商品信息：
```

- 类目: {category}
- 卖点: {selling_points}
- 价格区间: {price_range}
- 当前热搜词: {hot_keywords}

```
要求：
```

1. 生成 5 个标题，每个 25-30 字
2. 必须包含 2 个高搜索词
3. 突出"急" "便宜" "包教" "案例多"中的至少 1 个
4. 不能出现违禁词: {forbidden_words}
5. 输出 JSON 格式: [{"title": "...", "keywords": [...], "score": 0-10}]

```
直接输出 JSON，不要解释。
```

```
"""
```

```
class ContentAgent:
```

```
    def generate(self, sku):
```

```
        # 1. 标题
```

```
        titles = self.llm.invoke(TITLE_PROMPT.format(**sku.dict()))
```

```
        # 2. 主图 (ComfyUI 工作流)
```

```
        images = []
```

```
        for prompt in MAIN_IMAGE_PROMPTS[sku.category]:
```

```
            img = self.sd.generate(
                prompt=prompt,
                negative="低质量，模糊，水印",
                lora="electronic_commerce_v2",
                size=(800, 800)
            )
```

```
        # 自动加文案
```

```
        img = self.add_text_overlay(img, titles[0]["title"])
```

```
        images.append(img)
```

```
        # 3. 详情页(分段生成)
```

```
        detail = {
```

```
            "痛点": self.llm.gen("用户的 3 个核心痛点是?", sku),
```

```
            "服务": self.llm.gen("我们提供什么解决方案?", sku),
```

```
            "案例": self.fetch_cases(sku.category), # 从案例库抽3个
```

```
            "流程": STANDARD_FLOW,
```

```
            "价格": self.gen_pricing_table(sku),
```

```
            "FAQ": self.llm.gen("生成20个常见问题及答案", sku)
```

```
        }
```

```
        return Draft(titles=titles, images=images, detail=detail)
```


主图生成工作流 (ComfyUI) :

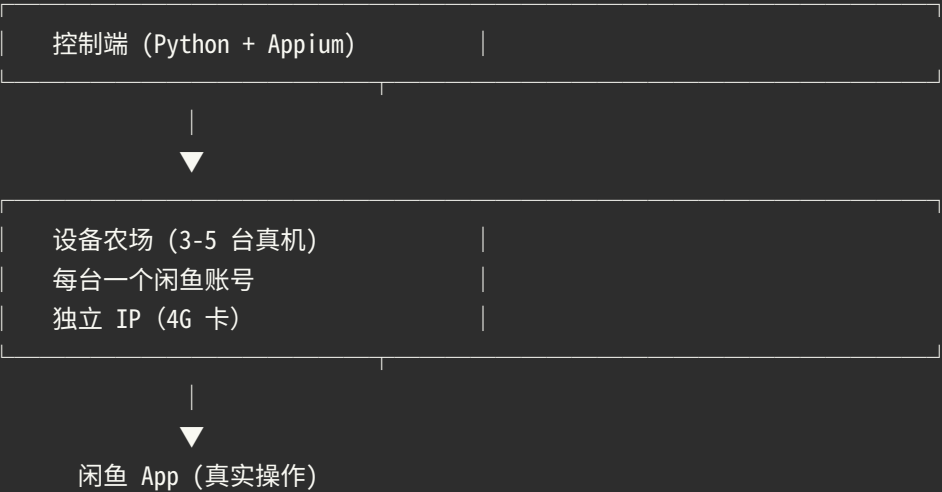
- SD 底模(electronic_commerce)
- LoRA(简洁风/科技风/3D 风随机)
- ControlNet(布局模板)
- 后处理(加文字/边框/二维码占位)
- 输出 800x800 PNG

自动化程度: 100% 全自动。每天凌晨 3 点跑一次，跑完邮件你。

智能体 ③ 自动上架 Agent (重点 · 防风控)

这是最难的一步，闲鱼对自动化检测严格，必须做"拟人化操作"。

架构:



防风控核心 (这部分是 know-how) :

```

class XianyuPostAgent:
    def post_item(self, draft):
        # 1. 拟人化时间分布
        sleep(random.uniform(30, 180)) # 不规则间隔

        # 2. 模拟真实用户行为路径
        self.scroll_homepage(seconds=random.randint(20, 60))
        self.tap_random_item() # 假装看了别人
        self.go_back()

        # 3. 进入发布
        self.tap("发布")
        sleep(random.uniform(2, 5))

        # 4. 上传图片(每张间隔随机)
        for img in draft.images:
            self.upload_image(img)
            sleep(random.uniform(3, 8))

        # 5. 输入标题(模拟打字速度)
        self.type_human_like(draft.titles[0],
                             min_delay=0.1, max_delay=0.4)

        # 6. 详情页分段输入
        for paragraph in draft.detail.split("\n"):
            self.input_detail(paragraph)
            sleep(random.uniform(0.5, 2))

        # 7. 价格(随机末尾数:99/88/66)
        price = self.smart_pricing(draft.sku)

        # 8. 提交前再随机等待
        sleep(random.uniform(5, 15))
        self.tap("发布")

        # 9. 记录指纹
        self.db.log_post(device_id, account, sku, time)

```

多账号矩阵策略： - 一台设备 = 一个账号 = 一张 4G 流量卡 - 每个账号每天最多 5 个新品 + 10 次擦亮 - 账号间内容差异化（同 SKU 用不同 AI 重写文案） - 24 小时内不在不同账号发同款（用文本相似度去重）

自动擦亮逻辑：

```
def auto_refresh(self):
    items = self.get_my_items()

    # AI 决策：擦亮哪些
    decision = self.llm.invoke(f"""
    我有这些商品: {items}
    曝光数据: {data}
    今天能擦亮 10 次，应该擦哪些？
    优先擦曝光下降但有过咨询的。
    """)

    for item in decision.targets:
        sleep(random.uniform(60, 300)) # 间隔 1-5 分钟
        self.refresh_item(item)
```

智能体 ④ 智能客服 Agent (RAG + 自主回复)

这是真正的 7×24 小时自动赚钱的关键。

架构：

新咨询消息

↓

[消息分类器]

- └─ 询价/砍价 → 推送给你 (高决策)
- └─ 询周期/源码 → AI 直接回复 (RAG 库)
- └─ 是否能做 X → AI 检索能力库 + 决策
- └─ 闲聊/无意向 → AI 礼貌回复 + 标记低意向

RAG 知识库结构：

```
knowledge_base/
|—— 服务能力/
|   |—— 数据采集_FAQ.md (50 个 Q&A)
|   |—— 自动化脚本_FAQ.md
|   |—— ...
|—— 报价规则/
|   |—— 定价表.json
|   |—— 优惠规则.json
|—— 案例库/
|   |—— case_001.md (含截图链接)
|   |—— ...
|—— 售后政策.md
|—— 引流话术.md
```

核心代码：

```

class CustomerServiceAgent:
    def on_new_message(self, msg, customer):
        # 1. 加载客户历史
        history = self.db.get_history(customer.id)

        # 2. 意向度评分 (0-100)
        intent_score = self.score_intent(msg, history)

        # 3. RAG 检索
        context = self.vector_db.search(msg, top_k=5)

        # 4. 决策树
        if intent_score > 80:
            # 高意向 → 引流
            reply = self.llm.invoke(f"""
            客户消息: {msg}
            客户历史: {history}
            知识库: {context}

            意向度高, 请生成引流话术, 逻辑:
            1. 先回答他的问题
            2. 暗示需求复杂需要详聊
            3. 引导加微信 (用暗语: VX/v/微信)
            """)
            self.send_reply(reply)
            self.trigger_followup(customer, delay_hours=24)

        elif intent_score < 30:
            # 低意向 → 标准应答
            reply = self.llm.invoke(f"""
            客户消息: {msg}
            知识库: {context}
            生成简短礼貌回复, 不要主动推销。
            """)
            self.send_reply(reply)

        else:
            # 中意向 → AI 多轮对话
            reply = self.llm.invoke(SYSTEM_PROMPT + history + msg)
            self.send_reply(reply)

        # 5. 涉及报价 → 必须人工
        if any(kw in msg for kw in ["多少钱", "便宜", "砍价", "议价"]):
            self.notify_human(customer, msg, suggested_reply=reply)
            return # 等你确认再发

```

意向度评分模型:

```
HIGH_INTENT_SIGNALS = [
    "什么时候能做完", "源码给吗", "可以加微信吗",
    "我有 X 个网站", "急用", "明天要", "怎么付款"
]

LOW_INTENT_SIGNALS = [
    "随便问问", "看看", "考虑一下", "太贵了"
]

def score_intent(msg, history):
    base = 50
    for sig in HIGH_INTENT_SIGNALS:
        if sig in msg: base += 15
    for sig in LOW_INTENT_SIGNALS:
        if sig in msg: base -= 10
    if len(history) > 5: base += 20 # 多轮对话
    if "?" in msg: base += 5 # 主动提问
    return min(100, max(0, base))
```

智能体 ⑤ 微信引流自动化

闲鱼侧 → 微信侧的衔接：

```

class TrafficAgent:
    def send_wechat_to_xianyu_buyer(self, customer):
        # 1. 闲鱼侧用暗语（规避平台检测）
        encoded = self.encode_wechat(self.my_wechat)
        # 例: "VX:zhang_san_2024" → "ⓧⓧ:zhang_san_2024"
        # 或拆字: "我的V是zhang underscore san"

        msg = f"""
        您好这个需求建议详细聊下，
        加我{encoded}，备注"闲鱼-数据采集"，我把案例发您看。
        """

        self.xianyu.send(customer.id, msg)

        # 2. 微信侧准备接收
        self.wechat_bot.set_pending(customer.id, expire_hours=48)

    def on_wechat_friend_request(self, request):
        # 3. 自动通过好友
        if "闲鱼" in request.note or "数据" in request.note:
            self.wechat.accept(request)

        # 4. 自动发欢迎语
        welcome = self.llm.gen_welcome_msg(
            buyer_history=self.find_xianyu_buyer(request)
        )
        self.wechat.send(request.user_id, welcome)

        # 5. 自动打标签
        self.wechat.tag(request.user_id, [
            "闲鱼来源",
            f"意向-{customer.last_sku.category}"
        ])

        # 6. 推送 3 张案例图
        for case in self.find_relevant_cases(request):
            self.wechat.send_image(request.user_id, case)

```

朋友圈 AI 自动发布：

```

class MomentsAgent:
    def daily_post(self):
        # 每天 3 条
        # 09:00 - 案例展示
        # 12:00 - 干货分享
        # 19:00 - 客户反馈截图

        case = self.fetch_latest_case()
        copy = self.llm.invoke(f"""
生成朋友圈文案，案例：{case}
要求：
- 100 字内
- 有钩子(数字/痛点)
- 末尾引导咨询
- 不刷屏感
""")

        self.wechat.post_moment(
            text=copy,
            images=case.screenshots[:9]
        )

```

智能体 ⑥ 客户运营 Agent（复购挖掘）

自动跟进未成交：

```

def followup_pending_customers():
    # 找 24-48 小时没回复的高意向客户
    pending = db.query("""
        SELECT * FROM customers
        WHERE intent_score > 70
        AND last_contact < NOW() - INTERVAL 24 HOUR
        AND status = 'pending'
    """)

    for c in pending:
        # AI 生成个性化跟进
        msg = llm.invoke(f"""
客户：{c.name}
上次聊到：{c.last_topic}
他没回复，写一条不打扰的跟进消息
要求：提供价值，不催单
""")
        wechat.send(c.id, msg)

```


90 天复购触达:

```
def repurchase_reminder():
    # 90 天前成交的老客户
    customers = db.query("""
        SELECT * FROM orders
        WHERE created_at BETWEEN
            NOW() - INTERVAL 90 DAY AND NOW() - INTERVAL 89 DAY
    """)

    for c in customers:
        # 个性化复购话术
        msg = llm.invoke(f"""
            老客户 {c.name}, 3 个月前买过 {c.last_sku}。
            现在我们升级了 {c.last_sku} 的功能, 包括 {NEW_FEATURES}。
            生成一条问候 + 升级介绍话术, 不要太硬。
        """)
        wechat.send(c.id, msg)
```

智能体 ⑦ 数据分析 Agent (自动决策)

全自动周报 + 自动调价:

```

class AnalyticsAgent:
    def weekly_decision(self):
        # 1. 采集数据
        data = self.collect_all_data()

        # 2. AI 分析
        insights = self.llm.invoke(f"""
        本周数据: {data}

        请分析:
        1. 哪些 SKU 转化率下滑超 20%? 建议下架还是改主图?
        2. 哪些 SKU 询单暴涨? 建议加价还是加供给?
        3. 哪些时段流量最好? 调整发布时间?
        4. 哪些关键词带来高意向流量? 加大投入?

        输出 JSON 决策:
        {{
            "下架": [...],
            "调价": [{{"sku": ..., "新价格": ..., "原因": ...}}],
            "加大投入": [...],
            "改主图": [...]
        }}
        """)

        # 3. 自动执行 (在你授权范围内)
        for action in insights.actions:
            if action.type == "调价" and action.delta_pct < 20:
                self.xianyu.update_price(action.sku, action.new_price)
            elif action.type == "下架":
                self.notify_human(f"建议下架: {action.sku}") # 等你确认
            # ...

        # 4. 生成 PDF 周报发邮件给你
        self.generate_pdf_report(data, insights)
        self.send_email("weekly_report.pdf")

```

五、部署方案

单服务器部署（推荐起步）

```
# docker-compose.yml
services:
  postgres:      # 数据库
  redis:         # 缓存 + 任务队列
  chroma:        # 向量数据库
  fastapi:       # 主业务 API
  celery:        # 定时任务
  ai-agent:      # AI 调度
  comfyui:       # 主图生成 (GPU 可选)
  monitor:       # 实时监控面板
```

最低配置：2 核 4G + 50G SSD $\approx$ ¥80-150/月（阿里云轻量） **推荐配置：**4 核 8G + GPU（用于 SD） $\approx$ ¥600/月（按需开关）

设备端（闲鱼操作）

配置	数量	单价	说明
二手安卓真机（红米/荣耀）	3-5	¥300-500	必须真机，模拟器易封
4G 流量卡（不同运营商）	3-5	¥30/月	一机一卡一 IP
USB Hub + 控制电脑	1	¥200	集中管理

总硬件成本：¥1500-3000（一次性投入）

六、月成本估算（全自动跑起来）

项目	月成本
云服务器	¥150
AI API（GPT-4o + DeepSeek 混用）	¥200-500
Stable Diffusion GPU（按需开）	¥100
4G 流量卡 × 3	¥90
域名 + 备案	¥10
合计	¥550-850/月

对比人工：1 个全职运营月薪 ¥6000-10000，省 90%。

七、4 周从 0 到 1 落地路线

第 1 周：基础设施

- ☐ 服务器搭建（Docker Compose）
- ☐ 数据库 schema 设计
- ☐ AI API 接入（GPT/Claude/DeepSeek）
- ☐ 跑通 1 个 SKU 的内容生成

第 2 周：上架自动化

- ☐ Appium 环境（1 台真机）
- ☐ 拟人化操作脚本
- ☐ 主图生成工作流（ComfyUI）
- ☐ 跑通：1 键发布 1 个 SKU 到闲鱼

第 3 周：智能客服

- ☐ RAG 知识库构建
- ☐ 意向度评分模型
- ☐ 接入闲鱼客服消息（轮询/Webhook）
- ☐ 微信侧自动化（企业微信优先）

第 4 周：调度 + 监控

- [] Celery 定时任务全部上线
- [] 实时监控大屏（看在跑的任务）
- [] 数据采集 + 周报生成
- [] 异常告警（账号被风控/AI 出错）

第 4 周末：你只需要每天打开监控面板看一眼，AI 已经自己卖货了。

八、风险与对冲（必读）

闲鱼平台风险

风险	概率	后果	对冲方案
自动化检测	中	限流/封号	拟人化 + 多账号矩阵 + 真机
引流违规	高	商品下架	暗语 + 不在主图详情留联系方式
价格频繁变动	低	限流	AI 调价限制日内 ≤2 次
同质化判定	中	限流	AI 重写 + 不同主图

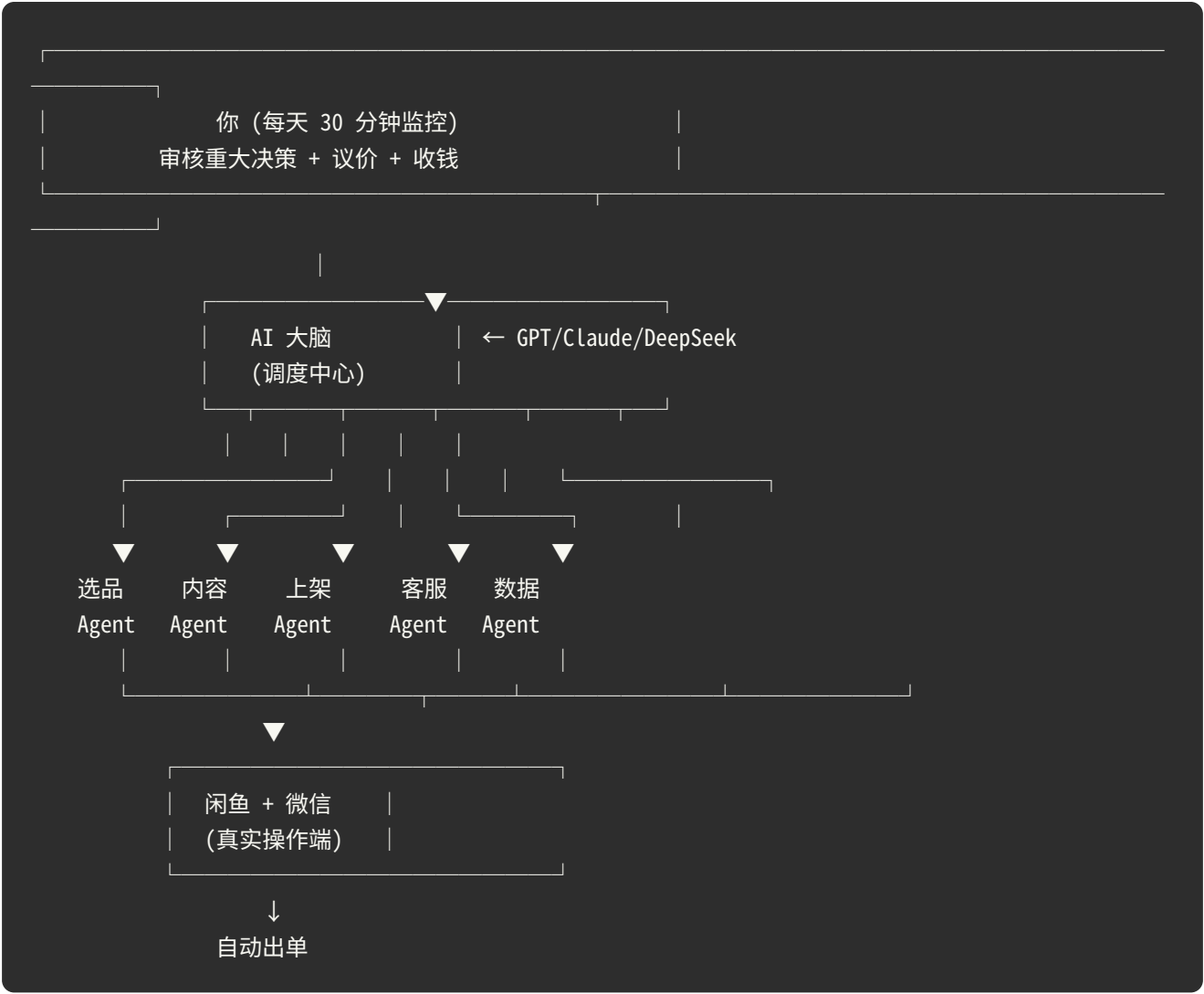
微信侧风险

风险	对冲
加好友过快	限制每日 ≤30 个 + 间隔随机
群发被封	不群发，1v1 个性化
ItChat 风险	优先用企业微信 API

法律合规

- 你卖的是合法服务
- AI 生成内容标注（可选）
- 不得做：刷单、虚假评价、诱导差评、绕过平台支付
- 不得做：未授权的数据采集、爬大厂、突破风控

九、最终架构总结图



核心思想：AI Agent 处理 90% 重复决策，你只决策 10% 关键节点。

效果预期：单店 GMV 月 ¥10000-50000，纯利润 60%+，每天投入时间 < 1 小时。

最后更新：2026 年 5 月